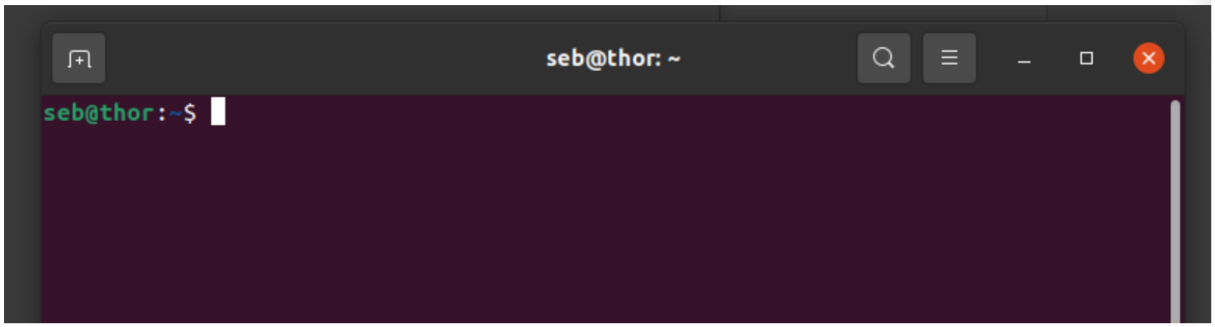
Lancez des commandes sous Bash

Utilisez le prompt et consultez ses variables

Une fois passées les étapes de connexion/authentification via une console ou un terminal, vous vous retrouvez devant cette interface assez austère qui vous présente un **prompt** :



Cette interface, le prompt, affiche déjà quelques informations intéressantes. Sur l'illustration ci-dessus par exemple, on voit que :

- je suis connecté avec l'utilisateur `seb` sur un équipement dont le "hostname" est `thor` ,
- je me situe actuellement dans le répertoire personnel de l'utilisateur, symbolisé par le caractère `~` .

Le dernier caractère `$` est le "delimiter". Il n'est pas innocent non plus : il m'indique ici que je suis un utilisateur non privilégié.

Avec le compte `root` , le caractère serait un `#` .

Enfin, en fonction des distributions, un curseur, éventuellement de couleur et/ou clignotant, vous indique la position actuelle de la saisie.

Toute cette configuration n'est pas magique. Elle est stockée dans un fichier et dans une variable gérée par le shell Bash. En l'occurrence, cette variable est nommée `$PS1` .

Je vous propose de consulter et de modifier cette variable pour adapter votre prompt. Et puis on en profite au passage pour s'intéresser à une seconde variable `$SHELL` , qui vous dit sur quel shell vous êtes connecté. C'est parti !

Plus d'informations sur le détail de la commande « **echo $PS1** » :
https://ss64.com/bash/syntax-prompt.html

Color Codes (ANSI Escape Sequences)

Foreground colors, Normal (non-bold) is the default, so the `0;` prefix is optional.

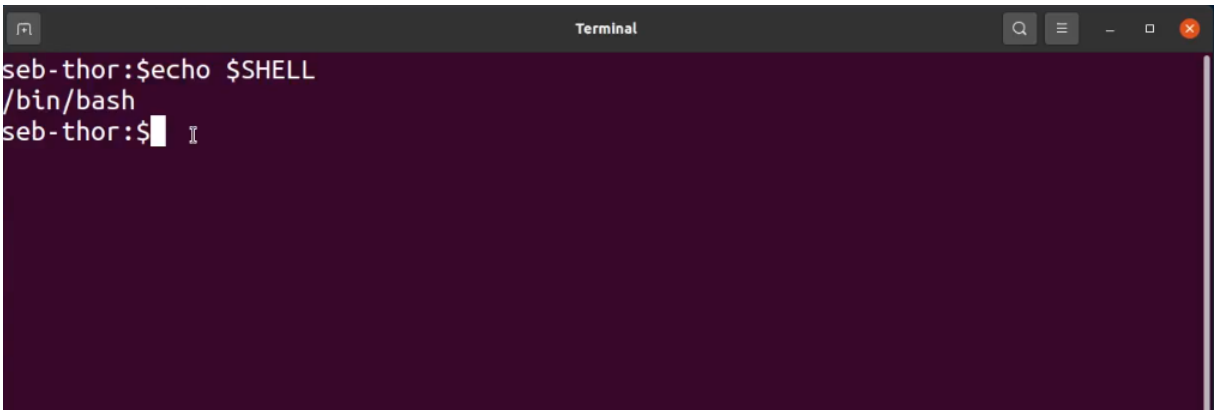
`\e[0;30m` = Dark Gray
`\e[1;30m` = Bold Dark Gray
`\e[0;31m` = Red
`\e[1;31m` = Bold Red
`\e[0;32m` = Green
`\e[1;32m` = Bold Green
`\e[0;33m` = Yellow
`\e[1;33m` = Bold Yellow
`\e[0;34m` = Blue
`\e[1;34m` = Bold Blue
`\e[0;35m` = Purple
`\e[1;35m` = Bold Purple
`\e[0;36m` = Turquoise
`\e[1;36m` = Bold Turquoise
`\e[0;37m` = Light Gray
`\e[1;37m` = Bold Light Gray

Background colors:

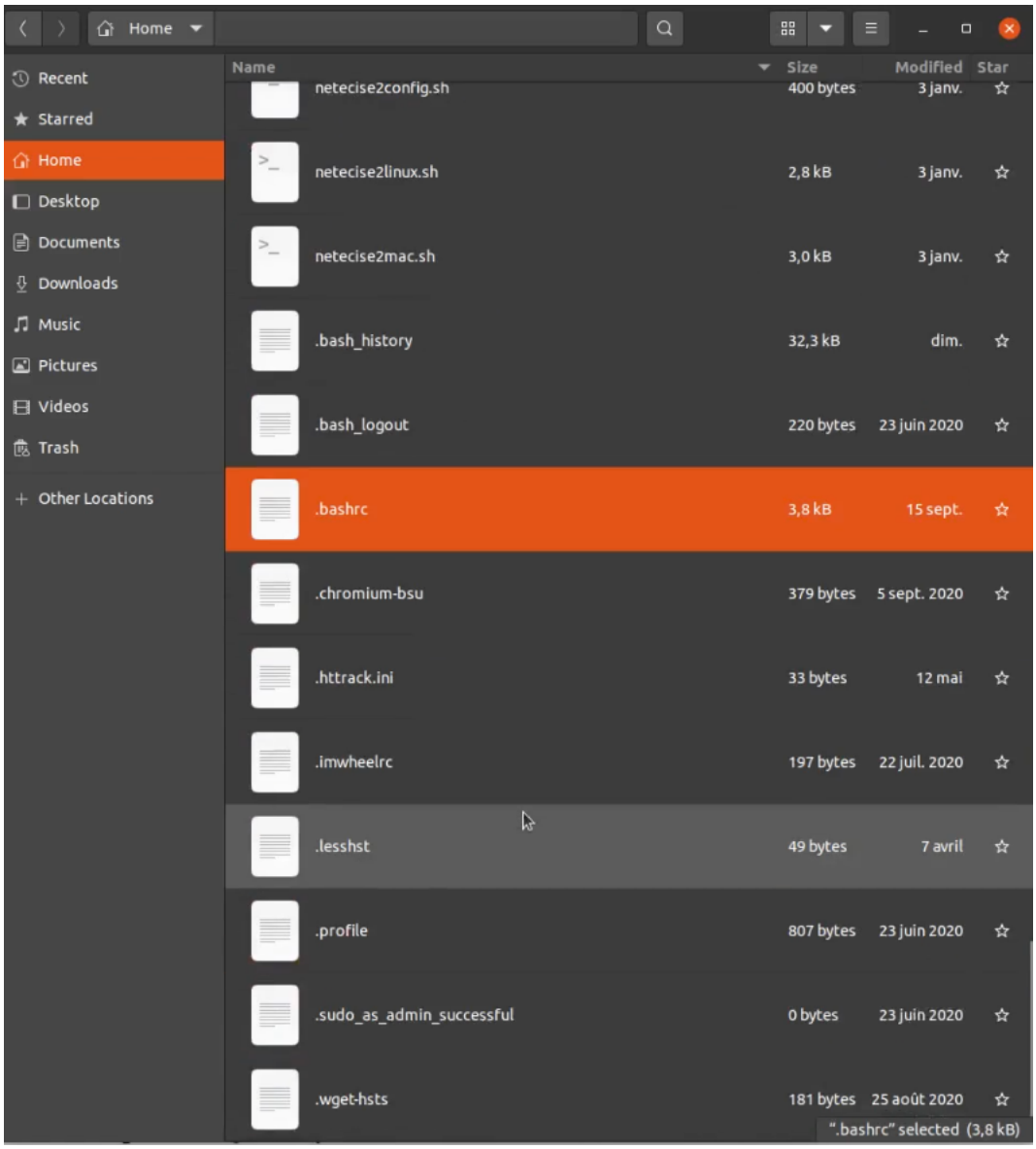
`\e[40m` = Dark Gray
`\e[41m` = Red
`\e[42m` = Green
`\e[43m` = Yellow
`\e[44m` = Blue
`\e[45m` = Purple
`\e[46m` = Turquoise
`\e[47m` = Light Gray

L'exécution des variables $PS1 et $SHELL :



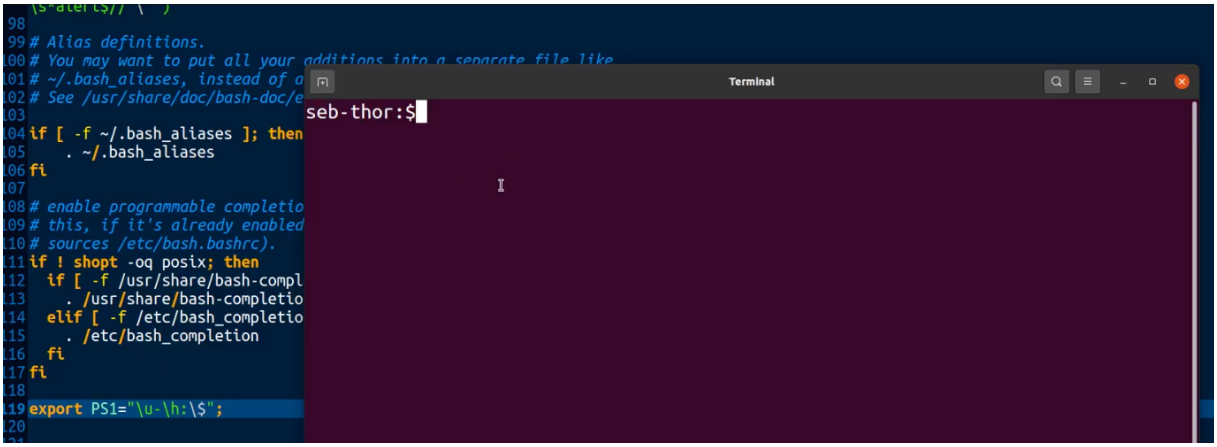


Garder la modification du prompt : accéder au chemin /home/user/.bashrc :

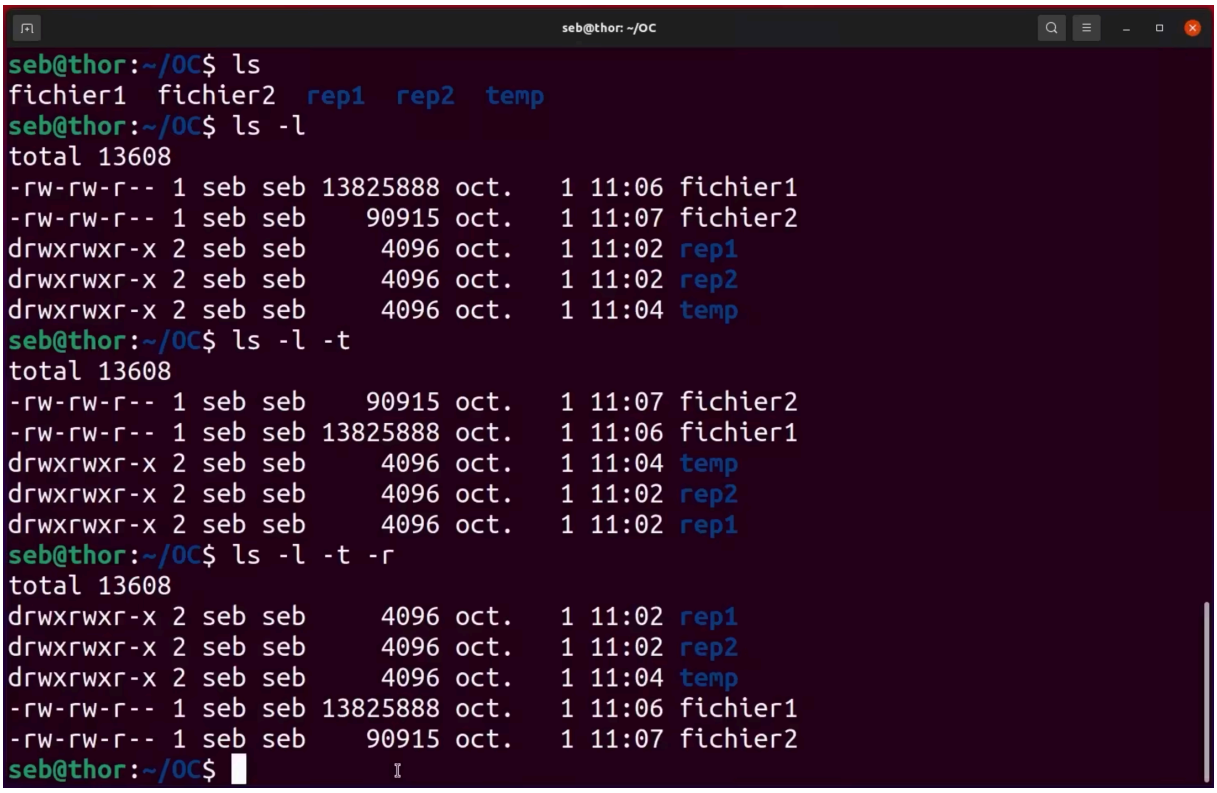


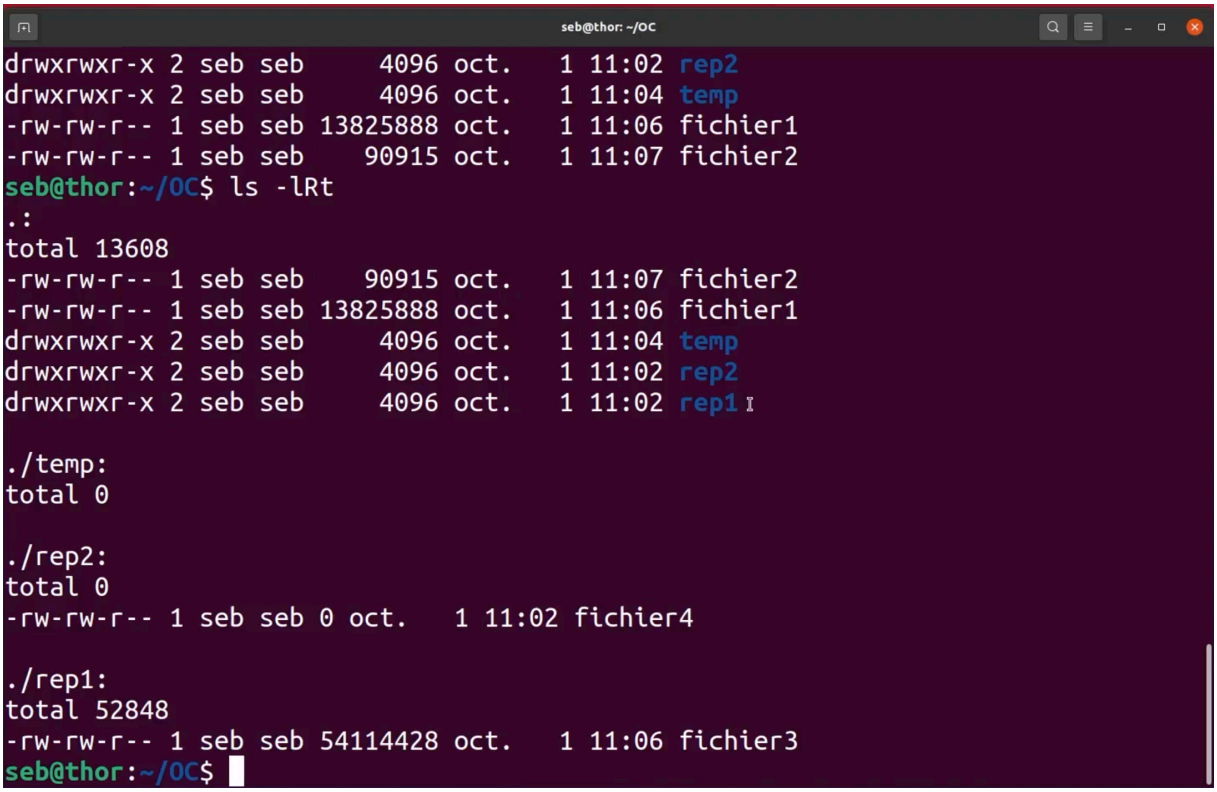
Ajouter à la fin du fichier .bashcr la ligne suivante et enregistrer:

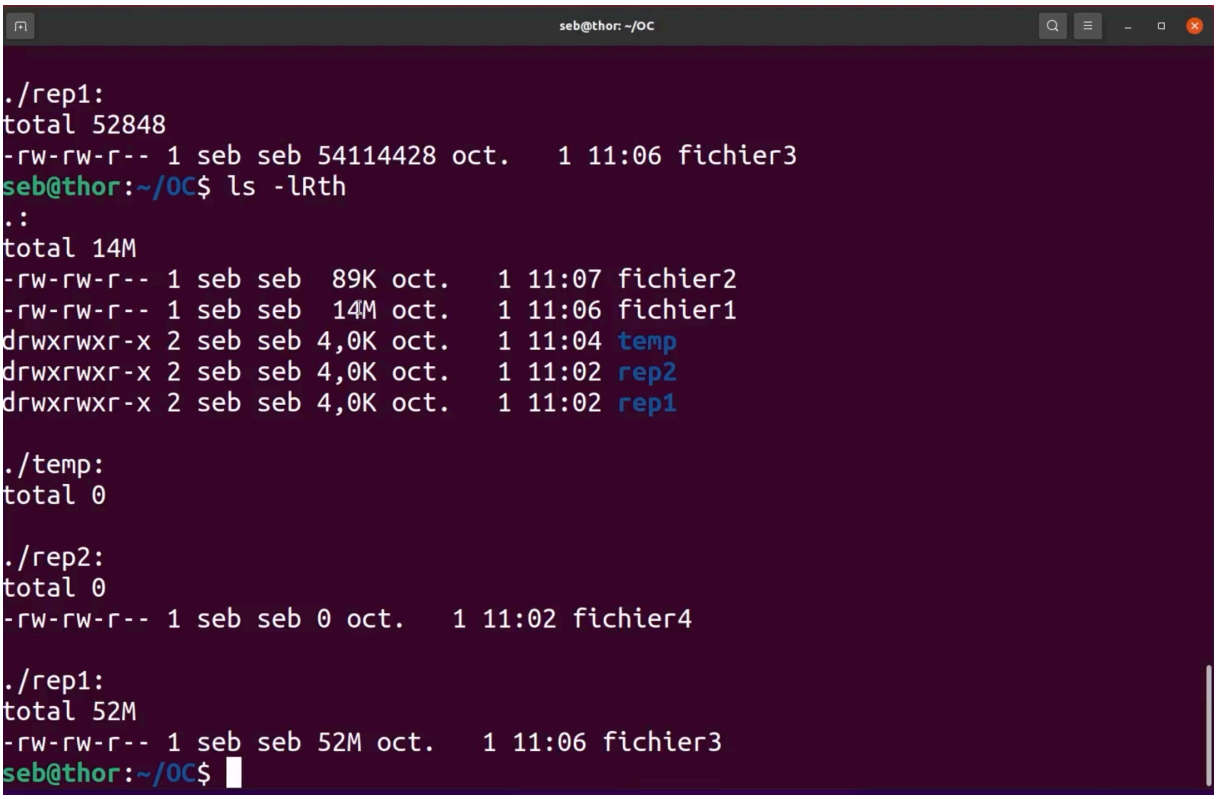
```
98
99 # Alias definitions.
100 # You may want to put all your additions into a separate file like
101 # ~/.bash_aliases, instead of adding them here directly.
102 # See /usr/share/doc/bash-doc/examples in the bash-doc package.
103
104 if [ -f ~/.bash_aliases ]; then
105     . ~/.bash_aliases
106 fi
107
108 # enable programmable completion features (you don't need to enable
109 # this, if it's already enabled in /etc/bash.bashrc and /etc/profile
110 # sources /etc/bash.bashrc).
111 if ! shopt -oq posix; then
112   if [ -f /usr/share/bash-completion/bash_completion ]; then
113     . /usr/share/bash-completion/bash_completion
114   elif [ -f /etc/bash_completion ]; then
115     . /etc/bash_completion
116   fi
117 fi
118
119 export PS1="\u-\h:\$";
120
```

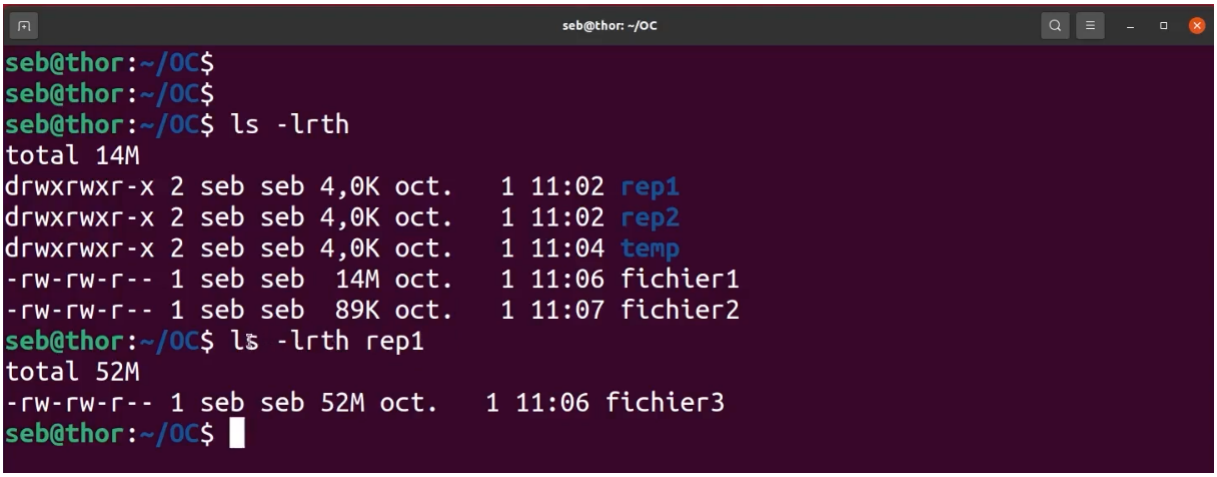


Listez le contenu d’un répertoire


seb@thor: ~/OC
seb@thor:~/OC$ ls
fichier1 fichier2 rep1 rep2 temp
seb@thor:~/OC$ ls -l
total 13608
-rw-rw-r-- 1 seb seb 13825888 oct.   1 11:06 fichier1
-rw-rw-r-- 1 seb seb   90915 oct.   1 11:07 fichier2
drwxrwxr-x 2 seb seb   4096 oct.   1 11:02 rep1
drwxrwxr-x 2 seb seb   4096 oct.   1 11:02 rep2
drwxrwxr-x 2 seb seb   4096 oct.   1 11:04 temp
seb@thor:~/OC$ ls -l -t
total 13608
-rw-rw-r-- 1 seb seb   90915 oct.   1 11:07 fichier2
-rw-rw-r-- 1 seb seb 13825888 oct.   1 11:06 fichier1
drwxrwxr-x 2 seb seb   4096 oct.   1 11:04 temp
drwxrwxr-x 2 seb seb   4096 oct.   1 11:02 rep2
drwxrwxr-x 2 seb seb   4096 oct.   1 11:02 rep1
seb@thor:~/OC$ ls -l -t -r
total 13608
drwxrwxr-x 2 seb seb   4096 oct.   1 11:02 rep1
drwxrwxr-x 2 seb seb   4096 oct.   1 11:02 rep2
drwxrwxr-x 2 seb seb   4096 oct.   1 11:04 temp
-rw-rw-r-- 1 seb seb 13825888 oct.   1 11:06 fichier1
-rw-rw-r-- 1 seb seb   90915 oct.   1 11:07 fichier2
seb@thor:~/OC$


seb@thor: ~/OC
drwxrwxr-x 2 seb seb   4096 oct.   1 11:02 rep2
drwxrwxr-x 2 seb seb   4096 oct.   1 11:04 temp
-rw-rw-r-- 1 seb seb 13825888 oct.   1 11:06 fichier1
-rw-rw-r-- 1 seb seb   90915 oct.   1 11:07 fichier2
seb@thor:~/OC$ ls -lRt
.:
total 13608
-rw-rw-r-- 1 seb seb   90915 oct.   1 11:07 fichier2
-rw-rw-r-- 1 seb seb 13825888 oct.   1 11:06 fichier1
drwxrwxr-x 2 seb seb   4096 oct.   1 11:04 temp
drwxrwxr-x 2 seb seb   4096 oct.   1 11:02 rep2
drwxrwxr-x 2 seb seb   4096 oct.   1 11:02 rep1
./temp:
total 0
./rep2:
total 0
-rw-rw-r-- 1 seb seb 0 oct.   1 11:02 fichier4
./rep1:
total 52848
-rw-rw-r-- 1 seb seb 54114428 oct.   1 11:06 fichier3
seb@thor:~/OC$



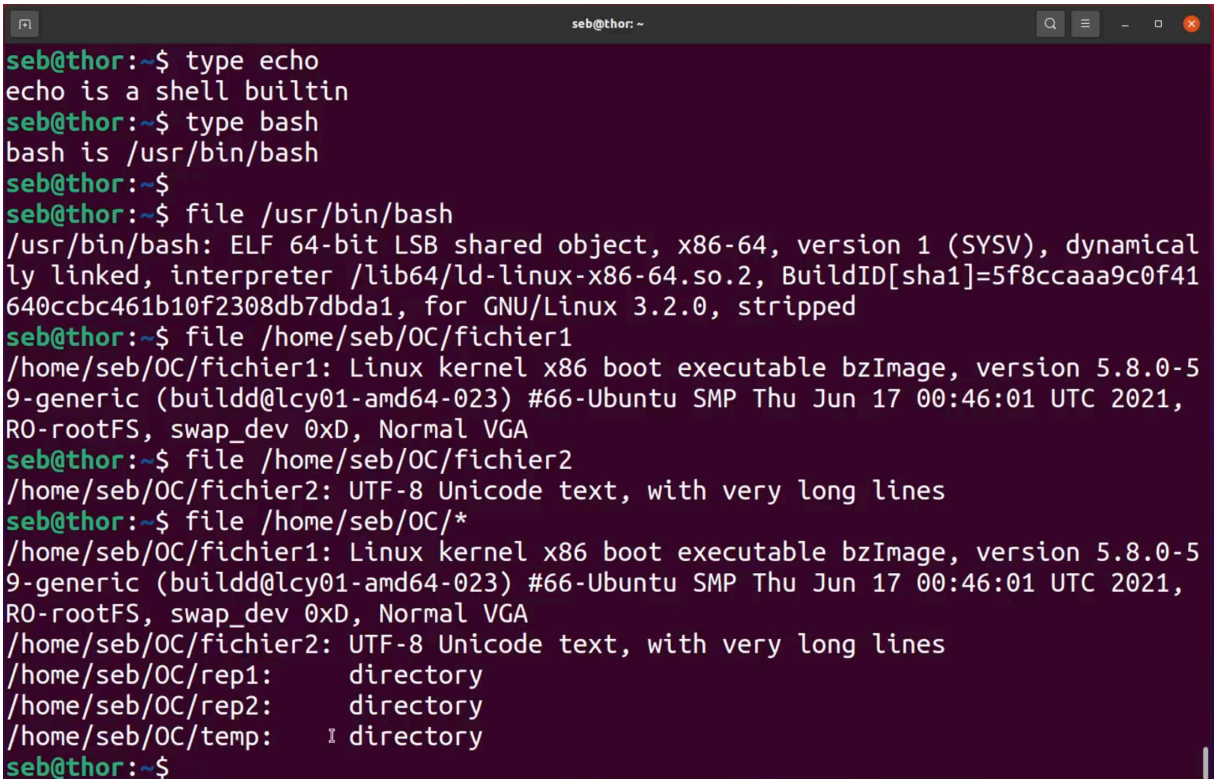


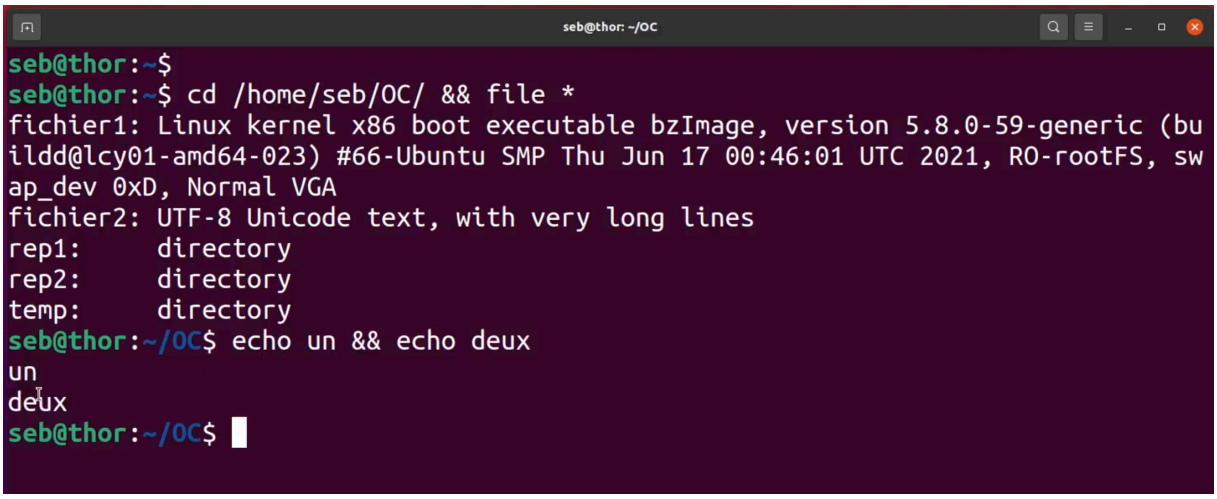
Comprenez les différents types de commandes Linux

Maintenant que vous connaissez la structure générale des commandes Bash, je vous propose de détailler les deux types de commandes disponibles sur le système.

- Les **commandes internes** : ces commandes sont livrées avec l'interpréteur de commandes directement, comme une primitive du shell. Elles sont disponibles à tout moment par l'administrateur du système. Cependant, il est probable que toutes ne soient pas forcément intégrées à l'interpréteur de commandes que vous allez utiliser. Il est également possible que leurs options ne soient pas toutes identiques non plus.
- Les **commandes externes** : ces commandes sont externes à l'interpréteur de commandes, c'est-à-dire qu'elles ne sont pas fournies par le shell. Elles sont généralement présentes sur le système sous la forme d'un fichier compilé binaire ou d'un fichier disposant des droits d'exécution (comme un script Perl par exemple).

Je vous propose d'étudier ces deux types de commandes. Vous découvrirez les nouvelles commandes **type**, **cd**, **id** et **file**, et au passage on en profite pour apprendre à chaîner les commandes.


seb@thor: ~
seb@thor:~$ type echo
echo is a shell builtin
seb@thor:~$ type bash
bash is /usr/bin/bash
seb@thor:~$
seb@thor:~$ file /usr/bin/bash
/usr/bin/bash: ELF 64-bit LSB shared object, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=5f8ccaaa9c0f41640ccbc461b10f2308db7dbda1, for GNU/Linux 3.2.0, stripped
seb@thor:~$ file /home/seb/OC/fichier1
/home/seb/OC/fichier1: Linux kernel x86 boot executable bzImage, version 5.8.0-59-generic (buildd@lcy01-amd64-023) #66-Ubuntu SMP Thu Jun 17 00:46:01 UTC 2021, RO-rootFS, swap_dev 0xD, Normal VGA
seb@thor:~$ file /home/seb/OC/fichier2
/home/seb/OC/fichier2: UTF-8 Unicode text, with very long lines
seb@thor:~$ file /home/seb/OC/*
/home/seb/OC/fichier1: Linux kernel x86 boot executable bzImage, version 5.8.0-59-generic (buildd@lcy01-amd64-023) #66-Ubuntu SMP Thu Jun 17 00:46:01 UTC 2021, RO-rootFS, swap_dev 0xD, Normal VGA
/home/seb/OC/fichier2: UTF-8 Unicode text, with very long lines
/home/seb/OC/rep1: directory
/home/seb/OC/rep2: directory
/home/seb/OC/temp: directory
seb@thor:~$


seb@thor: ~/OC
seb@thor:~$
seb@thor:~$ cd /home/seb/OC/ && file *
fichier1: Linux kernel x86 boot executable bzImage, version 5.8.0-59-generic (buildd@lcy01-amd64-023) #66-Ubuntu SMP Thu Jun 17 00:46:01 UTC 2021, RO-rootFS, swap_dev 0xD, Normal VGA
fichier2: UTF-8 Unicode text, with very long lines
rep1: directory
rep2: directory
temp: directory
seb@thor:~/OC$ echo un && echo deux
un
deux
seb@thor:~/OC$

```
seb@thor: ~/OC
seb@thor:~$
seb@thor:~$ cd /home/seb/OC/ && file *
fichier1: Linux kernel x86 boot executable bzImage, version 5.8.0-59-generic (bu
ildd@lcy01-amd64-023) #66-Ubuntu SMP Thu Jun 17 00:46:01 UTC 2021, RO-rootFS, sw
ap_dev 0xD, Normal VGA
fichier2: UTF-8 Unicode text, with very long lines
rep1:     directory
rep2:     directory
temp:     directory
seb@thor:~/OC$ echo un && echo deux
un
deux
seb@thor:~/OC$ id
uid=1000(seb) gid=1000(seb) groups=1000(seb),4(adm),24(cdrom),27(sudo),30(dip),4
6(plugdev),120(lpadmin),131(lxd),132(sambashare)
seb@thor:~/OC$ type id
id is hashed (/usr/bin/id)
seb@thor:~/OC$ file /usr/bin/id
/usr/bin/id: ELF 64-bit LSB shared object, x86-64, version 1 (SYSV), dynamically
 linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=f9dd79717ec20e1d
2cb2e593de4e92ba308feb69, for GNU/Linux 3.2.0, stripped
seb@thor:~/OC$
```

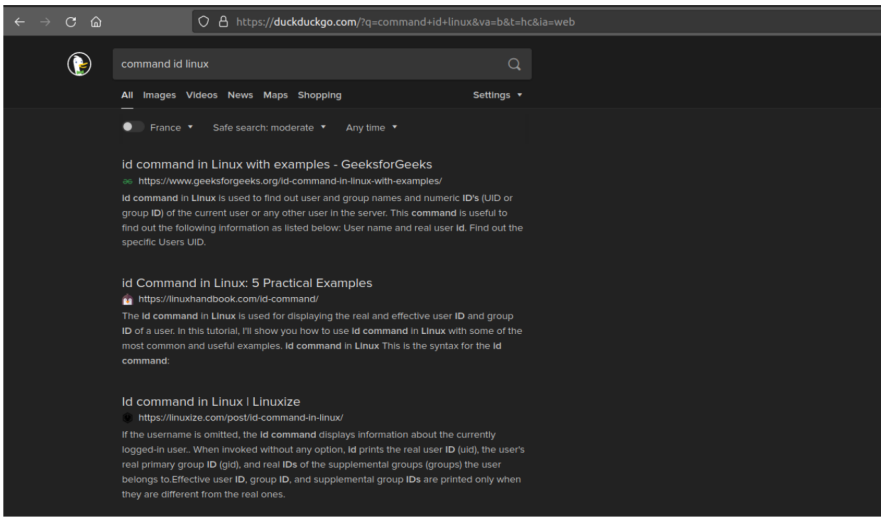
Consultez la documentation des commandes sous Linux

L'administrateur Linux dispose d'une panoplie assez conséquente de commandes en tout genre pouvant l'aider dans ses actions. Et même après plus de 25 ans de Linux derrière moi, je ne connais pas le fonctionnement de toutes ces commandes par cœur.

Heureusement, la qualité de la documentation Linux est l'une de ses grandes forces. Et les sources d'information sont multiples également.

Bien entendu, le Web et les moteurs de recherche restent vos amis lorsque vous bloquez sur une commande.

Dans la majorité des cas, il vous suffit de saisir le nom de la commande et votre objectif pour obtenir une liste de résultats probants.



Résultats sur l'affichage du groupe d'un compte avec la commande id

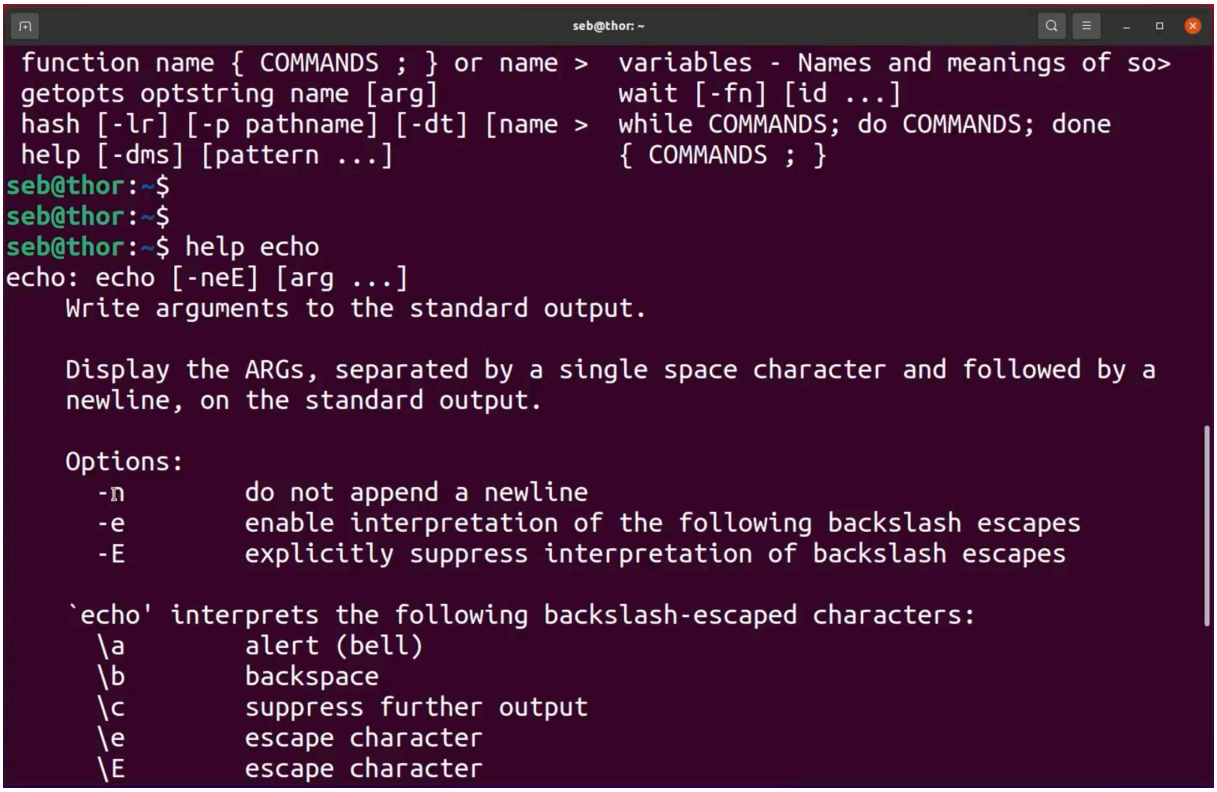
L'avantage principal de cette documentation réside dans le fait qu'elle soit accessible directement depuis le terminal, sans accès web.

Bon, il est vrai que le terminal n'est pas l'interface idéale pour consulter de la documentation, mais avec un peu de pratique, vous découvrirez les bonnes méthodes pour l'exploiter correctement.

Dans le screencast ci-dessous, je vous propose de consulter la documentation native des deux types de commandes :

1. internes
2. externes

En effet, en fonction de leur type, la documentation des commandes n'est pas accessible de la même manière. Et on en profite aussi pour découvrir ensemble le **manuel** officiel des systèmes Unix/Linux et la très célèbre commande qui l'accompagne : **man**.



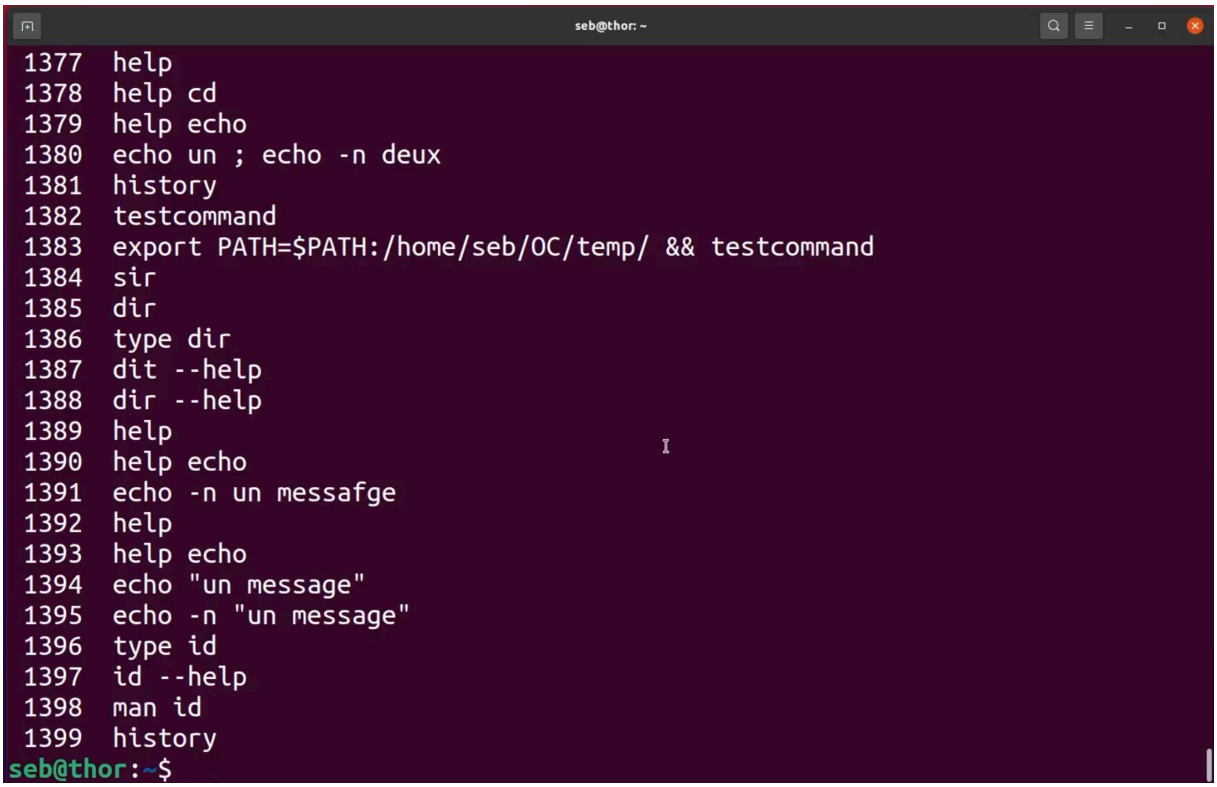
Profitez des fonctionnalités avancées de Bash

Vous voilà déjà super calé sur le terminal et les commandes du shell ! Dans cette dernière section, je vous propose de pousser le bouchon un peu plus loin…

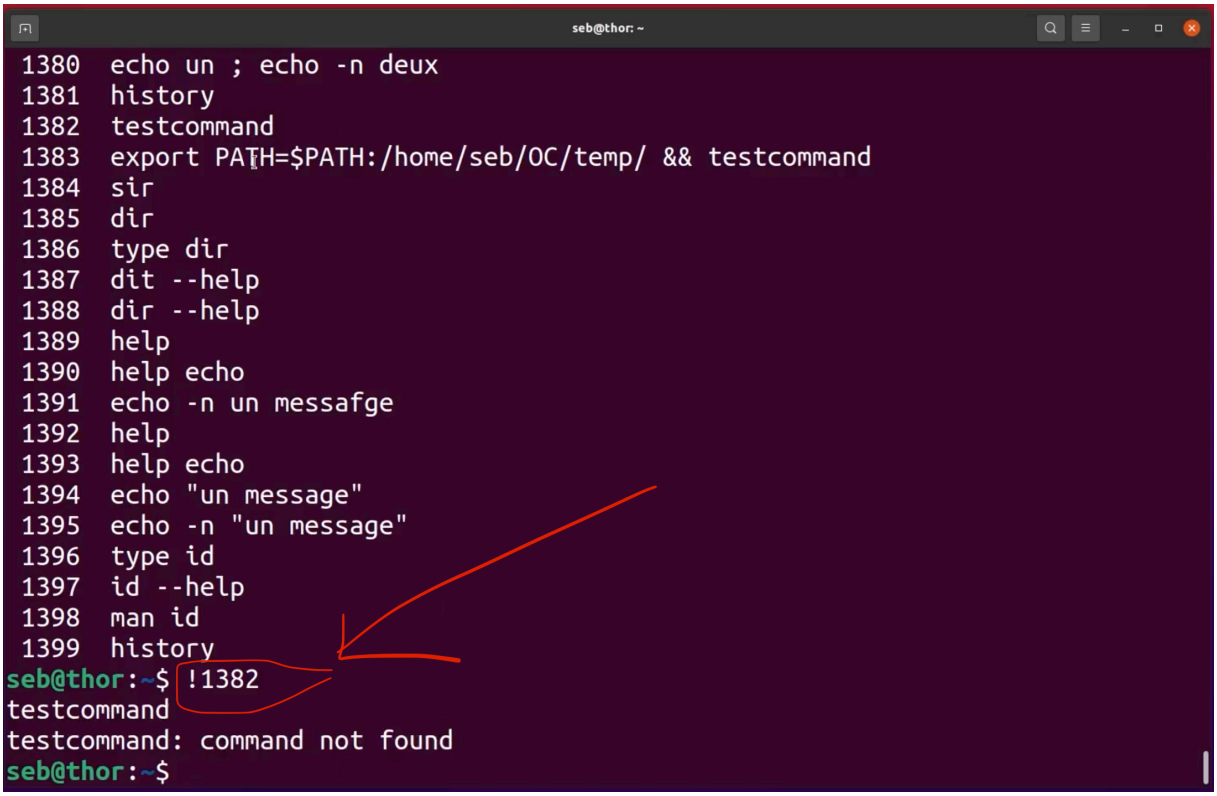
Je vous présente, dans ce screencast, deux fonctionnalités avancées de Bash qui participent aussi à faire de lui le shell de référence sous Linux. Et croyez-moi, vous allez très rapidement adopter ces fonctionnalités car elles sont très pratiques pour optimiser votre utilisation des commandes !

Ces deux fonctionnalités sont très intéressantes, mais leur fonctionnement est lié au terminal ou à la session courante.

Utiliser la commande **history**



Accéder à une commande dans l'historique des commandes exécutées :



En résumé

- La composition d'un prompt Linux et la variable **$PS1**(qui en définit chaque élément) n'ont plus de secret pour vous
- Vous avez compris la structure des commandes sous Linux, avec leurs options et leurs arguments
- Vous savez identifier les commandes internes et les commandes externes